

tmux documentation

§02 — Build Toolchain for Termux

Revision: 1 **Last modified:** 2026-05-22T07:20:02Z **Authority:** vasic-digital tmux project (research-only) **Maintainer:** milosvasic **Scope:** How to produce a tmux build and the tmx-state Go binary that run on Termux

1. Three production paths

There are three viable build paths for native binaries that run inside Termux. Each has different operator costs and different fit with our current `scripts/build_native.sh` model.

Path	Where the build runs	Effort to wire	What we get
(a) On-device build	inside Termux, <code>pkg install build-essential clang make autoconf</code>	Lowest (uses tools that already exist in the environment)	Slow but always-fresh; matches <code>build_native.sh</code> ergonomics
(b) NDK cross-compile from Linux/macOS host	Android NDK r26+ from host	Medium (new env vars, toolchain selection)	Fastest, suitable for CI, requires NDK install
(c) Termux's own <code>termux-packages</code> recipes	upstream package farm	Highest (PR upstream, ride their release cadence)	We don't ship our own binary — operators just <code>pkg install tmux</code>

Path (c) is how Termux's standard tmux package ships today. We CAN piggyback on it for the base tmux 3.6a build IF we're willing to rely on whatever Termux ships and only deliver our wrapper + Go daemon. For the v1.0.9 features we want under our control (jemalloc preload, OOM guard, the tmx wrapper) we must do (a) or (b).

2. Path (a) — On-device build inside Termux

Easiest mental model:

```
pkg install build-essential clang make autoconf automake pkg-
    config bison
pkg install libevent libevent-dev libjemalloc libjemalloc-dev
    ncurses-dev
# go for the state daemon:
pkg install golang
# our project:
git clone <our-repo> ~/tmux-project
cd ~/tmux-project
```

```
git submodule update --init --recursive
bash scripts/setup.sh
```

What works without modification:

- `tmux/configure` accepts the `bionic-libevent` + `bionic-ncurses` combination. Termux's own `tmux` package proves this — see <https://github.com/termux/termux-packages/blob/master/packages/tmux/build.sh>.
- `go build -o scripts/tmx-state-bin ./scripts/tmx-state/...` works natively because Termux's `golang` package is built for `bionic`, so the resulting Go binary is `bionic-linked`.
- `make`, `autoconf`, `automake`, `bash` — all the build helpers our `build_native.sh` calls — run identically.

What needs adapting in our build pipeline:

- `scripts/install_deps.sh` currently has a `Darwin / Linux` branch. We'd add a third arm case `"$HOST_OS" in *) [ -n "$TERMUX_VERSION" ] && pkg_install_termux ;; esac`. The detector is the `$TERMUX_VERSION` env var set by Termux's login script.
- `scripts/setup.sh` runs `sudo` for Linux's `apt install` step. Termux's `pkg` is non-`sudo` because the app data dir is writable by the app's own UID. The script must not call `sudo` on Termux.
- `bashrc_snippet.template` already targets `$HOME/.bashrc` — same on Termux.

Build time on a Snapdragon 8 Gen 2 phone for our `tmux` + `libevent` + `jemalloc` set is ~5-8 min wall clock based on community reports for similar `tmux-from-source` builds in Termux (UNCONFIRMED: — need empirical measurement on our actual hardware before quoting).

3. Path (b) — NDK cross-compile from a host

Use when CI / release artefacts are needed without an Android device in the pipeline.

The canonical Go-on-Android incantation (verified from <https://dave.engineer/blog/2025/11/cross-compiling-go-android/>):

```
export ANDROID_NDK_HOME=/path/to/android-ndk-r26d
export ANDROID_API=24          # Android 7 Nougat — Termux's
                                minimum

# For arm64 (most modern phones):
export GOOS=android
export GOARCH=arm64
export CGO_ENABLED=1
export CC="$ANDROID_NDK_HOME/toolchains/llvm/prebuilt/$(uname -s
    | tr A-Z a-z)-x86_64/bin/aarch64-linux-android$
    {ANDROID_API}-clang"

go build -ldflags='-extldflags=-pie' -o tmx-state-bin ./scripts/
    tmx-state/...
```

Variant for armv7 (older devices) — `GOARCH=arm GOARM=7 + armv7a-linux-androideabi24-clang`. Variant for x86_64 (emulator) — `GOARCH=amd64 + x86_64-linux-android24-clang`.

3.1 Why `CGO_ENABLED=1`

A pure-Go binary built with `CGO_ENABLED=0` ships Go's own DNS resolver. That resolver hard-codes `/etc/resolv.conf` and falls back to `[::1]:53` when the file is absent — which is exactly what happens on Android, where DNS goes through Android's system service. Result: any network call from the pure-Go binary fails. Source: <https://dave.engineer/blog/2025/11/cross-compiling-go-android/>.

For `tmx-state` specifically this might NOT matter — the daemon reads/writes a local JSON state file and never hits the network. We could in principle ship a `CGO_ENABLED=0` static binary. But the moment we add an optional Crashlytics-style report-home or a multi-device-sync hook, CGO becomes mandatory. Recommend setting `CGO_ENABLED=1` from day one.

3.2 PIE — Position Independent Executable

Android 5.0+ refuses to load non-PIE executables (error: only position independent executables (PIE) are supported). Source: http://gopinaths.gitlab.io/post/position_independent_error_in_android/. With Go 1.21+ and the NDK clang as CC, the linker produces PIE by default; the explicit `-extldflags=-pie` shown above is belt-and-suspenders for older Go.

For C builds (tmux, libevent, jemalloc):

- Configure with `--enable-pie` where supported.
- Add `CFLAGS="-fPIE -fPIC"` and `LDFLAGS="-pie"` when building manually.
- Termux's own `termux_step_pre_configure` injects these for every package — proof that the toolchain emits PIE binaries cleanly. See <https://github.com/termux/termux-packages/wiki>.

3.3 API level choice

The Termux Wiki notes: "Termux requires Android 7.0 or higher" (since their 2024 reset). Android 7 = API 24. Therefore `ANDROID_API=24` is the floor; choosing higher (e.g. 31 = Android 12) restricts compatibility for no gain.

`ANDROID_API=24` is widely deployed and produces a binary that runs on every Termux-supported device.

4. Path (c) — Use Termux's own packaging system

`termux-packages` is the GitHub repo that builds Termux's official package set. Each package has a `build.sh` recipe; a build farm produces `.deb` files served via the F-Droid Termux repo.

To ship via this path we'd:

1. Fork `termux-packages` (or PR upstream).
2. Add a `packages/tmx/build.sh` recipe declaring
`TERMUX_PKG_HOMEPAGE=<our-repo>`, `TERMUX_PKG_VERSION=1.0.9`,

TERMUX_PKG_DEPENDS="tmux openssh openssl", dependencies on Go for the build, etc.

3. Wait for the build farm CI + Termux maintainer review.
4. Operators get our wrapper from `pkg install tmx` and the tmux dependency satisfies itself from the standard Termux package.

Trade-off: zero on-device build cost for the operator, but we lose our verification gate (the `tmx` binary is never on `PATH` unless `scripts/verify.sh` reports green per §103) because Termux's CI does not run our gates. Path (c) is incompatible with our anti-bluff covenant as-is. We'd need to bring Layer 1 + 4 of §103 into the upstream recipe — substantial work.

Recommendation for now: skip path (c). Revisit if the port matures and Termux maintainers are receptive.

5. Go binary portability — `tmx-state` specifics

The v1.0.9 `tmx-state` daemon is `stdlib`-only with three sensitive syscalls:

Call	Linux support	Android (bionic) support
<code>os.OpenFile(O_CREAT O_RDWR O_EXCL)</code> for atomic temp-then-rename	works	works (Android Linux kernel underneath, same VFS)
<code>syscall.Flock(fd, LOCK_EX)</code> for cross-process state-file locking	works	works — <code>flock(2)</code> is a kernel syscall, bionic provides the wrapper
<code>f.Sync()</code> (<code>fsync</code>) for durability after <code>rename</code>	works	works

Verified via bionic status doc (<https://android.googlesource.com/platform/bionic/+HEAD/docs/status.md>) which lists `flock` and `fsync` as available since API 1. No special porting work needed.

`syscall.Setrlimit` is also supported on Android (bionic has architecture-specific `setrlimit.S` shims since the project's inception — Google-source ref above). Useful for §03's isolation discussion.

6. Build artefact location

On Termux there is no `/usr/local/bin`. We install our wrapper to `$PREFIX/bin/tmx` (the project's own `setup.sh` already chooses install location based on operator config — works as-is).

7. Multi-arch story

Modern phones (≥ 2020) are `arm64`. Older devices may still be `armv7`. `x86_64` only exists on emulators and some Chromebooks. We have three options:

1. **Single arm64 binary** — ship only `aarch64-linux-android`. Covers ~95% of in-the-field Termux installs (UNCONFIRMED — need Termux maintainer survey data). Operators on `armv7` build from source.
2. **Fat dual-arch tarball** — ship `arm64` + `armv7` side by side, runtime selector picks one. Doubles the wrapper-installer complexity.

3. **Source-only distribution** — operators always build via path (a). Simplest from our side; pushes work to the operator. Matches our current Linux/macOS posture (we don't ship a Mach-O binary today either).

Recommended path (3): source distribution. This is what `scripts/setup.sh` already does on Linux/macOS — adding a Termux arm is a straightforward extension.

8. Summary build matrix

Component	Linux today	macOS today	Termux future
tmux	from-source via <code>build_native.sh</code>	from-source via <code>build_native.sh</code>	from-source via <code>build_native.sh</code> (path a); OR <code>pkg install tmux</code> (path c hybrid)
jemalloc	<code>apt/dnf libjemalloc-dev</code>	<code>brew jemalloc</code>	<code>pkg install libjemalloc</code>
libevent	<code>apt/dnf</code>	<code>brew</code>	<code>pkg install libevent</code>
Go toolchain	<code>apt/dnf golang</code>	<code>brew go</code>	<code>pkg install golang</code>
tmx-state	<code>go build</code>	<code>go build</code>	<code>go build</code> natively, OR cross-compile with NDK
Wrapper	template-substituted shell	template-substituted shell	template-substituted shell

Build pipeline impact is contained: ~50 LOC of new branching in `install_deps.sh` and `setup.sh`, plus a new `$TERMUX_VERSION` detection guard at the top.

Sources

- <https://github.com/termux/termux-packages> — the official package recipes repo
- <https://github.com/termux/termux-packages/blob/master/packages/tmux/build.sh> — current Termux tmux recipe
- <https://github.com/termux/termux-packages/wiki/Common-porting-problems>
- <https://dave.engineer/blog/2025/11/cross-compiling-go-android/> — Go-on-Android cross-compile, including the DNS-resolver gotcha
- <https://android.googlesource.com/platform/bionic/+//HEAD/docs/status.md> — bionic API support matrix
- http://gopinaths.gitlab.io/post/position_independent_error_in_android/ — PIE requirement on Android 5+
- <https://ziggit.dev/t/native-android-bionic-build-on-termux-with-zig-0-15-2/14897> — community confirmation that bionic builds work end-to-end on Termux